Identifier Names in Computer Programs: Literature Review

Iwo Herka

University of Economics and Human Sciences in Warsaw - Computer Science

ABSTRACT

The current study aimed to conduct a literature review on the subject of identifier names in computer programs. Specifically, this review focused on three research topics: identifier structure and their semantics, identifier quality measures, and how developers choose names. This review included 33 papers extending from 1998 to 2021. Key findings suggest that lexicon quality and structure greatly impact developer cognition and performance, which translates to overall quality of software, as well as speed with which it is created. However, tools, methods and naming guidelines that improve the code lexicon are rarely applied commercially. Finally, future research is needed to develop a complete theory of program comprehension to understand the role of identifiers in the process.

KEYWORDS

programming
software development
psychology of programming
identifiers
naming

Contributions of this paper

- Summarises international research on identifier names in computer programs.
- Identifies methodological issues in the present research and offers recommendations for future studies.

INTRODUCTION

Existing models of program comprehension propose that comprehension occurs in a bottom-up manner, top-down manner or opportunistically, which combines two other schemes in some ad-hoc fashion. For example, Brooks (1983) argues that program understanding is fundamentally a top-down, hypothesis-driven approach in which an initial, low-resolution hypothesis is predicated on the developer's understanding of the program's domain. This initial assertion is subsequently refined into more precise hypotheses, drawing upon the information gleaned from the program's textual content and its associated documentation (Arnaoudova et al., 2014). On the other hand, the bottom-up approach recreates higher-level abstractions from lower-level artefacts through a reverse engineering process, which is essentially a mental pattern recognition (Rilling & Klemola, 2003). Other research has shown that, in fact, programmers often switch between these two models depending on the problem being solved (Shneiderman, 1976; Von Mayrhauser & Vans, 1998). However, as Arnaoudova et al. (2014) pointed out, regardless of which strategy programmers use, they spend a considerable amount of time reading program identifiers. Accordingly, many authors hint at the importance of identifier names in the process of program comprehension. Caprile and Tonella (2000) state that identifier names serve as a crucial source of insight regarding program entities. Deissenboeck and Pizka (2006) note that studies on the cognitive mechanisms of language and text comprehension indicate that the inherent semantics of words play a pivotal role in shaping the understanding process. Finally, Lawrie et al. (2007a) remark that for an engineer to understand a program, they must correlate the identifiers with the concepts they embody. Consequently, research on identifiers is not only vital to natural language program analysis (NLPA) and its applications but also to the overall improvement in programmers' productivity and code quality: to understand program comprehension, one must also study the role of identifier names in computer programs.

For this review, I identified the following main research areas which span the last three decades:

- Identifier structure and semantics. This line of research focuses on understanding how identifiers used by developers are structured and used to express meaning. Two main applications are to better understand how and why developers choose particular names and to help create methods and algorithms for NLPA (such as concept extraction), as well as methods for identifier processing, for example, splitting, expanding or tagging. Those, in turn, are used to create tools that are intended to help developers during program creation and maintenance. The primary methodology in this area is code analysis.
- Identifier quality measures. This area of investigation focuses on understanding the qualities of good and bad names and developing methods to measure identifier quality. Two main applications are to improve current code quality measures and help create tools and methods for improving names. The primary methodology is code analysis and experimental tasks with humans.
- How developers choose identifier names. This line of research is interested in understanding cultural, industrial, and cognitive processes that determine how and which names are used in

Corresponding author: Iwo Herka, University of Economics and Human Sciences in Warsaw - Computer Science, Okopowa 59 , Warsaw, 01-043 Poland.
Email: i.herka@vizja.pl

computer programs. One immediate application of this body of knowledge is to inform commercial software development processes and help guide future research. The methodology involves code analysis and experimental tasks.

METHOD

The goal of this review was to examine the current body of knowledge about identifier names in computer programs. The review consisted of peer-reviewed research studies published in computer science journals from 1998 to 2021, written in English.

The Search Process

The literature search was performed in two steps.

AUTOMATED SEARCH

The first step consisted of an automated search using five digital libraries and one broad indexing service: Computer Society Digital Library, ACM, Google Scholar, BASE, CORE, and SCOPUS. All searches were based on title, keywords, and abstract. The searches took place in December 2022. For all the sources except SCOPUS and IEEE, a set of simple search strings was used and aggregated. The searches were as follows:

- Google Scholar, BASE, and CORE:
  "software engineering identifiers"
  "software engineering identifier"
  "software engineering naming"
  "software engineering names"
  "software engineering code lexicon"

- SCOPUS:
  TITLE-ABS-KEY ("software engineering") AND TITLE-ABS-KEY ("identifier names" OR "names in programs" OR "code lexicon" OR "identifiers" OR "naming in programs")

- IEEE Computer Society Digital Library:
  "software engineering" in Anything AND ("identifiers" in Anything OR "identifier" in Anything OR "names" in Anything OR "naming" in Anything OR "code lexicon" in Anything)

- ACM:
  "software engineering" in Anything AND "identifiers" in Anything
  "software engineering" in Anything AND "identifier" in Anything
  "software engineering" in Anything AND "naming" in Anything
  "software engineering" in Anything AND "names" in Anything
  "software engineering" in Anything AND "code lexicon" in Anything

MANUAL SEARCH

Integrating the results of the different searches and screening the resulting papers by inspecting the title and keywords (if available) yielded 83 papers. This screening was based on excluding studies that were obviously irrelevant (not relating to identifiers or naming at all), duplicates, or papers which were previously found. In the subsequent phase, the papers that were selected during the previous stage were organised into a queue. Each individual paper was then systematically removed and its references were examined to identify any new, previously unencountered publications. Relevant papers were then appended to the queue, and this cycle continued until the queue was completely emptied and no further relevant papers were discovered.

Study Selection

The remaining publications were then subject to a more detailed assessment using the following inclusion/exclusion criteria:

- The paper must be on one of the following three topics:
  - Identifier structure and its semantics
  - Identifier quality measures
  - How developers choose names
- The paper must be a full publication (not a PowerPoint presentation or extended abstract)
- The paper must not be a personal account

Specifically because of the first requirement, technical papers relating to parsing, concept extraction, tokenization, and other language-processing or information-retrieval techniques were removed. This process yielded 33 final papers.

FINDINGS

Identifier Structure and its Semantics

The first formal attempt at analysing lexical, syntactic and semantic structure of function identifiers was made by Caprile and Tonella (1999). In their paper, they analysed the structure of function identifiers in C programs by parsing identifiers of nine public domain and one industrial programs using hand-crafted grammar and classifying them into six lexical categories: property check, transformation, indirect action, direct action, action on object, and double action. According to authors, the primary syntactic categories derived from the identifiers typically provide a significant overview of the function's behaviour, and the collection of associated key terms encompasses many fundamental elements that programmers commonly require to construct new identifiers. While they did not analyse specific grammar patterns, they examined grammatical components of function identifiers such as nouns and verbs and explored the various roles they play in expressing behaviour both individually and in combination with the proposed classes. Additionally, they demonstrated how functions may be classified according to domain ideas derived automatically from component nouns. Finally, they suggested ways for reorganising program identifiers using discovered lexical classes. Their work is similar to Anquetil and Lethbridge (1998), which examined filenames with the objective of grouping those that express similar concepts. However, while Anquetil and Lethbridge (1998) primarily focused on lexical elements, Caprile and Tonella (1999) also analysed syntactic considerations.

Shepherd et al. (2007) and Fry et al. (2008) were interested in methods for automatic extraction of natural language clues from source code (Natural Language Program Analysis). Their approach focused on identifying the entity (e.g., an object) which a specific verb (e.g., the action part of a method name) is targeting to better understand the behaviour of each method. To do so, they studied verb-direct objects (verb DO) to link verbs and entities they act upon and presented strategies for extracting them from method signatures and comments, as well as an empirical evaluation of their technique, which indicated 57% precision and 64% recall. Previous efforts in NLPA have centred around semi-automated techniques for associating design-level texts with code design patterns, as well as the utilisation of conceptual graphs to depict programs for code retrieval. However, this study represents a novel approach as it placed emphasis on uncovering high-level linguistic hints.

Høst and Østvold's (2009) work on identifiers begins with an examination of Java method names. They analysed a corpus of Java method implementations in order to deduce the meanings of verbs in method names from the methods' behaviour, which they quantified using a set of several method characteristics, such as whether it contains a loop, has specific return type, or throws an exception (Høst & Østvold, 2007). Using statistical analysis and information theory, they developed a vocabulary of commonly occurring verbs and their properties. This work is quite similar to work of Fry et al. (2008), except that the latter concentrated on starting verbs and method names, while the former focuses on class names and noun suffixes. Both papers indicate a growing trend towards utilising the knowledge present in identifiers.

Høst and Østvold (2009) extend the work by including whole method names, dubbed phrases, and a broader range of properties into their semantic model. As a result of their study, they were able to aggregate methods according to their phrases and extract their semantics using their semantic model, thereby modelling the link between method names and method activity. They looked at sentences that are very similar to the language patterns studied by Newman et al. (2020). Furthermore, the authors noted micro-patterns introduced by Gil and Maman (2005) as a significant source of inspiration for their work. These micro-patterns, which are machine-traceable patterns at the level of Java classes, have been hand-crafted by the authors to encapsulate their knowledge of Java programming. In contrast, Høst and Østvold (2009) utilised hand-crafted machine-traceable attributes to model the semantics of methods as opposed to classes.

Singer and Kirkham (2008) examined the relation between grammar patterns of class names and class properties called micro patterns. They identified the A*N+ (optional adjective followed by one or more noun) pattern as the main pattern used in languages like Java and C#. Working on this study was Butler et al. (2010), which studied identifier names and lexical inheritance, that is, how naming patterns propagate through inheritance hierarchies. They identified a number of additional grammar patterns: N+ (one or more noun), A+N+ (one or more adjective followed by one or more noun),, N+A+N+ (sequence of one or more noun, one or more adjective and one or more noun) and VN+ (verb followed by one or more noun). Additionally, they analysed Java properties, arguments, and variable name structures, finding that noun phrases are the most common and verb phrases are the second most common. Finally, they discussed Boolean variable phrase structures, noticing an increase in verb phrases relative to non-Boolean variables. The research undertaken by Høst and Østvold (2009) bears a resemblance to that of Singer and Kirkham. However, there are distinct differences in their methods of examination. Høst and Østvold concentrated on method names and initial verbs, while Singer and Kirkham focused on class names and noun suffixes. Furthermore, Høst and Østvold have produced automated documentation of pre-existing code conventions, while Singer and Kirkham have applied identified conventions to create developer-friendly tool assistants.

Gupta et al. (2013) described observed grammatical regularities in identifier names in their study on Posse, a part-of-speech tagger and syntactic chunker. Among other things, they found that function names that begin with the base form of a verb often exclude the entity/object that the action is intended to affect or that certain function names imply an activity (e.g., "elementAt" has an implicit get action). They noticed that noun phrases are normal for classes and attributes, but that Booleans are typically outliers. Their work is an extension of Høst and Østvold (2007), which cannot be utilised to tag arbitrary method signatures and are restricted solely to method names. In contrast, Gupta et al. (2013) showed that it is possible to effectively handle both common and uncommon naming conventions, without being constrained to just method names.

Newman et al. (2020) employed part-of-speech tagging techniques to analyse frequent grammar patterns used by developers. As a result, they generated a taxonomy of grammatical patterns obtained directly from the source code. They identified most commonly used grammar patterns such as NM+N (one or more noun modifiers followed by a noun) which is a noun phrase pattern (e.g. "factor class", "auth result", "previous caption"), V NM+N (sequence of a verb, one or more noun modifiers and a noun), which is a verb phrase pattern where a verb is followed by a noun phrase ("check gl support", "resize nearest neighbour"), or V* NM+ NPL (optional verb followed by one or more noun modifiers and plural noun), which is either a plural noun phrase pattern or a plural verb phrase pattern ("mkt factors", "num cols", "get raw arguments", "validate class rules"). They found that identifiers found in C, C++ and Java are largely similar. The majority of patterns that appear more than once in any language also exist in other languages. However, this finding should not be taken as a definitive answer for programming languages in general, especially where different paradigms are used. C, C++ and Java are all procedural, with varying degrees of support for object-oriented programming. The authors emphasise that their research complements the study by Arnaoudova et al. (2014), which centred on rename analysis. Specifically, their examination of naming structure can enhance the analysis of changes between two name versions, partly by bolstering the performance of standard part-of-speech taggers. Furthermore, grammatical patterns presented by Newman et al. (2020) were very similar to Høst and Østvold (2009)

Identifier Quality Measures

FORMAL MODELS

Deissenboeck and Pizka (2006) defined the first formal model for evaluation of correctness, consistency, and conciseness of identifiers names. In their model, somewhat simplifying, they recognize concepts, names for the concepts, program entities, and identifiers of program entities. Within the model, consistency, that is, absence of synonyms and homonyms, means that there is a 1:1 correspondence, that is, a unique relation, between concept names and concepts themselves. For example, the name "config_filename" concisely represents the concept of the name of the configuration file. On the other hand, correctness means that the identifier of a program entity that manifests a concept must correspond to the unique name of the concept or one of more general concepts. This rule makes sure that identifier names do not become completely meaningless or incorrect. For example, in a function called "p," which returns a permutation of a set of elements, because "p" is neither the name of the concept of permutation nor of a generalisation of it, the identifier violates the correctness rule. A wrong identifier, such as "file," would also be disqualified. However, correctness is not enough to make an identifier such as "transformation" an invalid identifier. It corresponds to the name of a generalisation of the concept, but it is only of limited assistance for the reader. This is a very common problem: identifiers are often correct, but not concise enough. Conciseness, therefore, requires that an identifier has exactly the same name as the concept it stands for, no generalisations allowed. In the above example, only "permutation" would be a correct and concise name for the function "p." To enforce those rules, the authors proposed the usage of a tool-supported identifier dictionary (IDD). This dictionary would contain a list of already-used identifiers and additional information such as the identifier's type and a developer's description. When in search of a name, one may next look at the existing identifiers. Additionally, the tool detects two inconsistencies: when identical IDs of various types exist in the same project and when identifiers are declared but never used. As of writing this paper, we were not aware of any commercial or open-source developer tools which would leverage methods proposed by the authors.

One downside to the approach proposed by Deissenboeck and Pizka (2006) is the requirement of manual mapping between identifiers and domain concepts (i.e., concept mapping). While this mapping may be generated concurrently with the program for new projects at a reasonably modest cost, for existing applications, the task may be exponentially more challenging and costly. In a response to Deissenboeck and Pizka (2006), Lawrie, Felid et al. (2006) and Lawrie et al. (2007b) developed a more easily automated method for verifying that identifiers are well-formed without any additional information, that is, by only considering a syntactic structure of identifiers. They noticed that a comparable grammatical structure suggests a breach of both the syntactic-synonym consistency criterion and the syntactic conciseness criterion. In each case, one identifier is encompassed within another. In result, an identifier A which is contained in identifier B, fails the consistency requirement (e.g., "position" and "relative_position"). Furthermore, if there exists a third identifier which also contains identifier A, it implies that program contains two distinct concepts, and that identifier A does not accurately represent either of them, thus failing conciseness requirement (e.g., "position", "relative_position" and "absolute_position"). Using a large corpus of source code (48 million lines), the authors' evaluation of the technique showed that 75% of detected syntactic violations would also be detected with manual concept mapping.

CATALOGUE-BASED METRICS

An alternative strategy for measuring the quality of identifiers is evaluation against lexicon guidelines. In this approach, identifiers are checked for compliance with a set of ad-hoc rules. The identifier naming guidelines constitute most of the advice found in programming literature and conventions around the structure and grammar of identifier names. An identifier is classified as of poor quality if it violates any of the guidelines. The rules also often take the form of "lexicon bad smells" (LBS) or linguistic antipatterns (LA) catalogues (we will refer to LBSs and LAs interchangeably). LA is a concept similar to that of a "code smell" or software antipattern (Brown et al., 1998), but refers to the structural problem of an identifier. An example of LA is when the term is used to name both the whole and its parts, such as when class "Account" also contains property "account." LAs can also be purely syntactic, for example, when the identifier is too short or too long. It is worth noting that the perception of LAs can be relative and their meaning and use contingent upon many factors, such as the programming environment, programmers' culture, or the peculiarities of the project. LA in one context cannot be considered LA in another. In the following paragraphs, we review studies related to guideline-based metrics.

Relf (2004) formulated 21 identifier style guidelines using Ada and Java. The guidelines focused primarily on relatively simple syntactic rules which can be checked without in-depth semantic analysis. Examples include using more than one underscore character in a row, using too short names, using at least two words, using only English words, or duplicating words in different order. As such, Relf's guidelines do not deviate significantly from official Java identifier naming conventions nor address problems of identifier conciseness and consistency and appropriateness. Relf (2005) extends the research from Relf (2004), which introduced identifier style guidelines to experimentally check whether programmers improve identifier quality with the help of a tool to dynamically offer feedback on their practices. The quality of identifiers was measured with 19 identifier-naming style guidelines defined in the previous study. Test subjects consisted of university students and professional programmers. Results showed that programmers who used dynamic feedback produced identifiers of higher quality.

Butler et. al. (2009) adapted 12 guidelines proposed in Relf (2004) to evaluate quality of identifiers in from eight open-source Java projects. Guidelines were selected as it pertains to the structure of identifier names and allows for objective measurement. In some cases, the guidelines were refined. For example, for the rule stating that identifiers should only consist of English words, the authors assembled a dictionary with approximately 117,000 words, encompassing inflections as well as American and Canadian spelling differences. This compilation was derived from word lists from the SCOWL package up to size 70. Additionally, around 90 prevalent computing and Java terms, such as 'arity', 'hostname', 'symlink', and 'throwable', were incorporated.. Results showed a statistically significant correlation between identifiers violating the guidelines and code quality issues detected by the FindBugs tool.

However, in Butler et al. (2009), the authors discussed how their previous findings were based on a somewhat limited perspective on identifier quality and were exclusively focused on Java projects. The author stated that in order to understand the effect of identifier quality on source code readability, it's essential to deepen our understanding of identifier name quality, considering aspects of typography, natural language usage, and grammar. Additionally, Butler et al. pointed out that work of Liblit et al. (2006) and Caprile and Tonella (2000) on identifier grammar can be applied to create better quality metrics. Finally, the authors proposed "to develop a metalanguage to describe identifiers in situ and their relationships with other adjacent identifiers in programming language elements, such as operators. Such a metalanguage would describe source code as the mixture of programming language and natural language identifiers that is perceived by the human reader, and may offer an-other perspective on the readability of source code, not just identifier name quality" (p. 34).

Butler et al. (2010) expanded their previous work by extending the analysis to the relation between identifier quality and Java methods. In this study, the authors evaluated eight open-source Java projects using 10 identifier guidelines and six source code metrics, including FindBugs, the three-metric maintainability index, a human-trained readability metric, and cyclomatic complexity. In result, the authors found that identifier names of subpar quality correlate strongly with increased complexity and decreased readability and maintainability in source code. Employing natural language and commonly recognized abbreviations in identifier names can serve as a rudimentary indicator of source code quality. The length of identifiers, considering both character count and the number of constituent words, can act as a basic classifier for assessing complexity and maintainability. Although there's a connection between low-quality identifier names and Find-Bugs warnings, the dynamics of this relationship are intricate and seem to be specific to the application. Notably, the sole negative correlations identified were present in commercialised projects, suggesting potential distinctions between open-source and commercial software.

Lawrie et al. (2007a) explored four hypotheses related to identifier quality by studying a large set of source code in several languages. In their study, covering almost 50 million lines of code, they defined identifier quality as the use of dictionary words, common programming terms or well-known abbreviations.

The first attempt at identifying and cataloguing LBSs (LAs) comes from Abebe et al. (2009b). Based on a collection of code smells introduced by Fowler and Beck (1997), the authors compiled a list of lexicon bad smells hosted on a Wiki. Unfortunately, as of writing this review, the webpage is unavailable. Their paper, however, described five selected bad smells which are: "odd grammatical structure" (e.g., method identifiers should start with a verb), "term used to name both the whole and its parts," "inconsistent identifier use" (all soft terms of an identifier are contained in the same order in another identifier of the same entity type and both identifiers are located inside the same container entity) and "useless type indication". It's notable that the last rule is very similar to the syntactic rule for conciseness and consistency introduced by Lawrie, Felid et al. (2006), Lawrie et al. (2007b). Secondly, the first rule ("odd grammatical structure") is very general and it is unclear exactly what should qualify as "odd" except for a few examples mentioned in the paper. Finally, to automatically detect LAs in source code, the authors developed a suite of tools, called LBSDetectors (which support 11 LAs).

Abebe et al. (2012) used previously defined LBSs to measure quality of identifiers, specifically to investigate whether using LBS in addition to structural metrics improves fault prediction. In order to do this, they compared the prediction capability of a model using only structural metrics and a model using structural metrics together with LBS-based metrics. The findings from three open source systems, ArgoUML, Rhino, and Eclipse, revealed that the majority of instances have improved. The authors noted that among all LBS, "overloaded identifiers" and "inconsistent terms" stand out as the most significant." In the majority of the systems, these are the major contributors of at least one retained principal component. They are also the most important contributors for fault prediction, as "synonym similar" which always ranks among the top five most important LBS. On the other hand, authors believe that other LBSs, not included in this list, should not be deemed irrelevant, as they become important for specific systems, for example, extreme contraction and misspelling.

In Arnaoudova et al. (2013; Arnaoudova, 2014), the authors coined the term "linguistic antipattern" as separate from "lexicon bad smell," highlighting the difference in the scope of those two terms. Lexicon bad smells primarily target individual identifiers. In contrast, LAs emphasise a more detailed perspective, addressing inconsistencies between method names, parameters, return types, and comments, as well as discrepancies between attribute names, types, and comments. The paper introduced a catalogue of LAs related to methods (i.e., when a method does more than it says, says more than it does, or does the opposite of what it says) and attributes (i.e., when an attribute contains more than it says, says more than it contains, and contains the opposite than it says). To help in detecting potential LAs in Java programs, authors prototyped the Linguistic Anti-Pattern Detector (LAPS) and tested it on four Java programs. The study reported a precision of 72% in detecting LAs.

OTHER METRICS

In Arnaoudova et al. (2010; Arnaoudova, 2014), two new metrics related to program identifiers - term entropy and context coverage - were introduced. Term entropy measures how scattered words contained in identifiers are across program constructs: "To compute the term entropy, we consider terms as random variables with some associated probability distributions. Given a term, its entries in the term-by-entity matrix are the counts of term occurrences and thus by normalising over the sum of its row entries a probability distribution for each term is obtained" (p. 36). Context coverage, on the other hand, measures how unrelated are the methods and attributes containing these terms and is calculated using Latent Semantic Indexing, an advanced information retrieval technique. The study presents a case study which shows that high entropy and high context coverage correlates with probability of it being faulty.

Recognizing that unusual code may be symptomatic of possible defects, Reiss (2007) suggested an automatic method for detecting such code. The strategy consisted of extracting common syntactic code patterns from a collection of programs and recognizing them as anomalous code that does not match these patterns. According to Høst and

Østvold (2009), this strategy is comparable to theirs, with the primary distinction being that Høst and Østvold defined atypical code in the context of a given method phrase. However, as noted by the authors, their current technique has limitations. Firstly, it is highly dependent on the selected samples. Programming teams, communities, programming languages, and paradigms may produce different, but valid, code patterns. Secondly, the current method finds patterns that are not in the pattern library, assuming that the pattern library is a reliable source of information. Lastly, the method only works for small code segments.

Thies and Roth (2010) devised a method for detecting specific inconsistency in naming, which leverages the fact that a variable assigned to another probably references the same objects, and when declared with an identical type, it is likely intended for a similar function. To test this observation, the authors created an automated tool to analyse assignments and type information to recommend refactorings. In their case study, 21 of 32 warnings were valuable, resulting in a success rate of 65.6%.

Fakhoury et al. (2018; Fakhoury et al., 2020) introduced a methodology for quantifying developers' cognitive load using eye tracking and functional near infrared spectroscopy (fNIRS), which measures blood oxygenation levels in the prefrontal cortex, on the assumption that "mentally demanding tasks require resources in the prefrontal cortex." Authors argue that "[fNIRS] can provide a minimally invasive way to empirically investigate the effects of source code on human cognition and the hemodynamic response within physical structures of the brain" (p. 289). A preliminary study concluded that by employing fNIRS and eye-tracking instruments, the cognitive load of developers can be precisely linked to identifiers in source code and text, demonstrating a 74% similarity to self-reported instances of high cognitive load.

How Developers Choose Names

Feitelson et al. (2020) performed a sequence of studies involving 334 individuals in which they selected names in various programming settings. All of the names in the experiment followed a three-step naming process: (a) choosing the concepts to represent, (b) choosing the words to describe each concept, and (c) assembling the name. Authors tested if explicitly using this approach may improve name quality. The results revealed that names created following the scheme outscored those chosen in the original experiment 2:1. Most notably, throughout 47 experiments, the median likelihood of two developers picking the same name was 6.9% but once a name was chosen, most developers understood it. This shows the process of choosing a name is highly individual and highly variable.

Lawrie et al. (2007a) investigated four hypotheses about the quality of identifiers by examining a vast collection (50 million lines) of code in several programming languages across 30 years. They aimed to answer whether modern programs manifest better identifier quality, whether identifier quality changes across program lifetime, whether programming language impacts identifier quality and whether identifier quality changes across proprietary and open-source programs. In relation to the first question, authors found a reduction in the need for hard word splitting in more modern programs and an increase in the percentage of dictionary soft words. In an identifier, hard words are strings of characters between word breaks (e.g. underscores or camel-casing). Soft hordes, on the other hand, consist of one or more hard words and identify a concept. In relation to the second question, studies showed that while identifier quality tends to improve with each version, this enhancement is not consistent across all cases and reaches a saturation point rapidly. Next, authors found that "the data support the observation that open source includes a higher percentage of dictionary words, while proprietary source code in-cludes more abbreviations (some known abbreviations, but mostly unknown abbreviations placed in the "other" category)" (p. 379). Finally, authors didn't find strong evidence that programming language has significant influence on the quality of identifiers.

Liblit et al. (2006), examine programming as a cognitive communication activity. They explore the criteria that influence the selection of individual names and how those names, specifically their grammatical structure, convey relevant information about program behaviour. For example, they show use of valence in method names to express when the method accepts one or more arguments, or how methods called for side-effects (modifying program state) are named using verb phrases in imperative mood.

They also discuss conflicting pressures impacting name length. They note that although longer names are more informative, they are also more challenging to spell and read. Additionally, they often exceed the display's width and become troublesome in larger expressions. Terse and shortened names may increase readability at the expense of expressiveness and are difficult to comprehend in the absence of domain knowledge on part of the programmer. Additionally, abbreviations may result in an overflow of names that are difficult to comprehend. Finally, they remark that programmatic names vary in terms of their semantic roles and visibility to other code, meaning that the need to be informative may be relaxed for names with a restricted reach or limited uses.

Furthermore, they argue that identifier names are often metaphors and that metaphors "[...] are central to naming program entities in a piece of software (Douce, 2004) as many studies on program comprehension have verified (Ben-Ari & Sajaniemi, 2004; Ehrlich & Soloway, 1984; Sajaniemi, 2002). Names can also be classified into groups, called concept keywords, and associated with programming metaphors used during the design of data abstractions" (Ohba & Gondow, 2005, p. 54). Indeed, Douce (2004) presents a long list of other metaphors found in computer programs.

Finally, they perform statistical analysis of identifiers in a number of large-scale applications. For example, in Windows 2003 Server code consisted of 45 million lines and contained more than 7 million identifiers, evenly divided between global and local definitions. Among others, authors showed that local names averaged about 7.6 characters in length and 1.8 subwords while global averaged 17.2 characters with 4.9 subwords. They note, however, that standard deviations are quite high and one must not draw detailed conclusions.

Gresta and Cirilo (2020) studies context similarity between names in computer programs. They found that for all projects examined, developers indeed use semantically similar names: "These results indicate that developers tend to always give good names to identifiers – ones that are at least a bit related between themselves. Only a small portion of

the files have a negative internal similarity. In these cases, we consider that developers are not using existing words; or are using abbreviations and completely unrelated words." Additionally, the paper concluded that "developers in most cases resort to words that exist in the dictionary". These findings confirm results presented in Lawrie et al. (2007a). Finally, authors found that "the number of changes in a file is a promising indicator to distinguish between files with distinct levels of internal similarity – there is a tendency that the more developers modify a file in a project, the more they improve identifiers similarity" (p. 56).

Abebe et al. (2009a) attempted to analyse how the source code lexicon evolves throughout program lifetime by studying two programs. Authors found that the vocabulary and size of the code tend to develop together, that most new identifiers don't add new words, but rather use existing ones and that problem domain keywords dominate class, attribute, and function names. Authors conclude that the development of source code vocabulary does not follow a simple pattern, and further study is required to completely understand it.

Haiduc and Marcus (2008) performed a case study of six software libraries to examine how domain terms are used in comments and identifiers. Authors found that "within the studied software, [...] on average: 42% of the domain terms were used in the source code; 23% of the domain terms used in the source code are present in comments only, whereas only 11% in the identifiers alone, and there is a 63% agreement in the use of domain terms between any two software systems" (p. 113). These results indicate that domain terms are used frequently in software projects and that comments and identifiers are sources of a significant fraction of domain terms. Finally, authors argued that "comments may be more significant than identifiers when it comes to reflect domain information" (p. 121).

DISCUSSION

Summary of Key Findings

The goal of this review was to identify and synthesise scientific literature on the topic of identifier names in computer programs. 9 out of 33 presented studies analysed syntactic and semantic structure of identifiers. Among others, authors identified common grammar patterns used by developers, developed vocabularies of frequently used verbs and their properties, developed methods to describe code behaviour and to detect naming bugs by analysis identifiers, studied relations between grammar patterns and discussed how developers choose names in cognitively motivated ways. 18 studies focused on the problem of measuring identifier quality. In this line of work, authors developed two main approaches to measuring identifier quality, that is, those using formal models such the one proposed by Deissenboeck and Pizka (2006) and catalogue-based metrics, which score lexicon by detecting linguistic antipatterns and naming bad smells. To that end, multiple such naming guidelines were proposed. Some of the alternative approaches proposed include physical and conceptual identifier dispersion and Reiss's method of pattern matching. Next, six papers took a look at how developers choose and use names in software projects. Among others, authors concluded that probability of two developers choosing the same name is very low, that modern programs have improved in lexicon quality, that there is no significant evidence that programming language influences lexicon quality and that less than half of all domain terms are present in the source code in the form of identifiers and comments.

Methodological Issues in Current Research

LIMITATIONS OF CONSENSUS-BASED METRICS

Using opinion-based or consensus-based metrics for measuring identifier quality can be problematic for several reasons. Firstly, these metrics are subjective and rely on the personal opinions or consensus of individuals or groups, rather than objective criteria. This can introduce bias into the measurement process and result in inconsistent or inaccurate assessments of identifier quality. Secondly, personal opinions and consensus can change over time, making these metrics prone to change and making it difficult to compare results across different studies or over time. Lastly, the opinions and consensus regarding identifier quality can vary across different communities, programming languages, and programming paradigms. This means that the same identifier may be considered good quality by one group of people, but poor quality by another. Using opinion-based or consensus-based metrics for measuring identifier quality can be unreliable, inconsistent, and prone to bias, making it difficult to draw meaningful conclusions about the quality of identifiers.

LIMITATIONS OF ANALYSIS-BASED STUDIES

Five out of six mentioned papers studying the process of choosing names by developers were based on analysis of already written code. Analysing already-written code to study how developers chose names for their applications is problematic for a number of reasons. First, code analysis does not include the context in which the code was created, including the social variables, organisational factors, language, tools of programming paradigm that may have influenced the naming conventions utilised. For instance, different organisations have different naming conventions, and developers may have to abide by them. Secondly, code analysis does not factor in the cognitive processes that developers go through when choosing names for their programs. For example, code analysis cannot capture how developers may have struggled to find an appropriate name, how they may have debated different options with their colleagues or how the names change throughout the peer-review process.

Code analysis is a useful tool but it's limited and doesn't factor in many variables that can influence the naming conventions, therefore, a more comprehensive approach that includes multiple perspectives such as interviews, surveys, version-control analysis and direct observations is needed to gain a better understanding of how developers choose names for their programs.

PRESENCE OF STUDENTS IN EXPERIMENTAL TASKS

The first issue pertains to the use of students as participants in experimental tasks. It is unclear how transferable results derived from

experiments involving students without professional experience are to the real-word software industry. Professional developers manifest different levels of proficiency in almost all programming tasks and it is not unreasonable to assume that experienced developers use different cognitive strategies in solving tasks such as code comprehension or bug localization (Beniamini et al., 2017; Lawrie, Felid et al. 2006; Lawrie et al., 2007b; Scanniello & Risi, 2013) are all examples of studies which used only students. Other papers, such as Arnaoudova et al. (2016), Relf (2005), and Salviulo and Scanniello (2014) use both students and professional programmers which may help to amortise the lack of experience on the part of the subjects.

USING TOY PROBLEMS

The previous problem may be magnified when experiments use real production code, instead of highly artificially constructed one (or where are not based on the source code at all). In the latter case, differences in productivity may be minimised, but also concerns about external validity are raised in regards to the code itself, as it less accurately exemplifies the actual engineer environment. For example, Binkley et al. (2009) designed a task in which subjects were read aloud a phrase and were instructed to locate an identifier written in one of two styles on the screen, moving around. As Binkley et al. (2009) points out: "selecting names from production code represents a compromise between control (e.g.,the high-control of artificially constructed source) and applicability of results to the code found in real programs" (p. 432).

SUBJECTIVE VERIFICATION METHODS

Third issue pertains to the use of verification methods based on self-report. As Fakhoury et al. (2018; Fakhoury et al., 2020) notes, most common measures of comprehension are subjective, e.g.: self-reports of perceived comprehension, think aloud protocols, surveys or comprehension summaries. Authors point out that "there are several potential issues that can result from using these indirect measures because they are prone to participant biases (Hochstein et al. 2005). For example, participants may report they understood a piece of source code, but their perceived understanding does not necessarily mean they correctly understood the source code." One example of this can be found in Binkley et al. (2019) which studied the impact of identifier style on code readability or in Lawrie, Morrell et al. (2006; Lawrie et al., 2007) which used source code summaries and self reported confidence levels were used to assess comprehension levels of participants reading source code snippets containing single letter, abbreviated, and full length identifiers (Fakhoury et al., 2019; Fakhoury et al., 2020).

CONCLUSION

In this review we presented a literature review on the topic of identifier names in computer programs. We identified and summarised following areas of research: identifier structure and their semantics, identifier quality measures, how developers choose names. We presented key findings, discussed methodological issues and finally suggested recommendations for future research.

ACKNOWLEDGEMENTS

The author reports no conflicts of interest. The author reports no sources of funding. The author would like to thank Krzysztof Rychlicki-Kicior, PhD, for this feedback and guidance throughout the work on this review.

DATA AVAILABILITY

No data associated with this publication was produced.

REFERENCES

Abebe, S. L., Arnaoudova, V., Tonella, P., Antoniol, G., & Guéhéneuc, Y. -G. (2012). Can lexicon bad smells improve fault prediction? In R. Oliveto, D. Poshyvanyk, J. Cordy, & T. Dean (Eds.), *2012 19th Working Conference on Reverse Engineering* (pp. 235–244). https://doi.org/10.1109/WCRE.2012.33

Abebe, S. L., Haiduc, S., Marcus, A., Tonella, P., & Antoniol, G. (2009a). Analyzing the evolution of the source code vocabulary. In R. Ferenc, J. Knodel, & A. Winter (Eds.), *2009 13th European Conference on Software Maintenance and Reengineering* (pp. 189–198). https://doi.org/10.1109/CSMR.2009.61

Abebe, S. L., Haiduc, S., Tonella, P., & Marcus, A. (2009b). Lexicon bad smells in software. In A. Zaidman, G. Antoniol, & S. Ducasee (Eds.), *2009 16th Working Conference on Reverse Engineering* (pp. 95–99). https://doi.org/10.1109/WCRE.2009.26

Anquetil, N., & Lethbridge, T. (1998). Extracting concepts from file names; a new file clustering criterion. *Proceedings of the 20th International Conference on Software Engineering* (pp. 84–93). https://doi.org/10.1109/ICSE.1998.671105

Arnaoudova, V. (2014). *Towards improving the code lexicon and its consistency* [Doctoral dissertation, École Polytechnique de Montréal]. PolyPublie. https://publications.polymtl.ca/1517/

Arnaoudova, V., Di Penta, M., & Antoniol, G. (2016). Linguistic antipatterns: What they are and how developers perceive them. *Empirical Software Engineering, 21*(1), 104–158. https://doi.org/10.1007/s10664-014-9350-8

Arnaoudova, V., Di Penta, M., Antoniol, G., & Guéhéneuc, Y.-G. (2013). A new family of software anti-patterns: Linguistic anti-patterns. In A. Cleve, F. Ricca, & M. Cerioli (Eds.), *2013 17th European Conference on Software Maintenance and Reengineering* (pp. 187–196). https://doi.org/10.1109/CSMR.2013.28

Arnaoudova, V., Eshkevari, L., Oliveto, R., Guéhéneuc, Y.-G., & Antoniol, G. (2010). Physical and conceptual identifier dispersion: Measures and relation to fault proneness. *2010 IEEE International Conference on Software Maintenance* (pp. 1–5). https://doi.org/10.1109/ICSM.2010.5609748

Arnaoudova, V., Eshkevari, L. M., Penta, M. D., Oliveto, R., Antoniol, G., & Guéhéneuc, Y. -G. (2014). REPENT: Analyzing the nature of identifier renamings. *IEEE Transactions on Software Engineering, 40*(5), 502–532. https://doi.org/10.1109/TSE.2014.2312942

Ben-Ari, M., & Sajaniemi, J. (2004). Roles of variables as seen by CS educators. *SIGCSE Bulletin, 36*(3), 52–56. https://doi.

org/10.1145/1026487.1008013

Beniamini, G., Gingichashvili, S., Orbach, A. K., & Feitelson, D. G. (2017). Meaningful identifier names: The case of single-letter variables. *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)* (pp. 45–54). https://doi.org/10.1109/ICPC.2017.18

Binkley, D., Lawrie, D., Maex, S., & Morrell, C. (2009). Identifier length and limited programmer memory. *Science of Computer Programming, 74*(7), 430–445. https://doi.org/10.1016/j.scico.2009.02.006

Binkley, D. W., Davis, M., Lawrie, D. J., & Morrell, C. (2019). To CamelCase or under_score. 2019 *IEEE/ACM 27th International Conference on Program Comprehension (ICPC)* (pp. 177–177). https://doi.org/10.1109/ICPC.2019.00035

Brooks, R. (1983). Towards a theory of the comprehension of computer programs. *International Journal of Man-Machine Studies, 18*(6), 543–554. https://doi.org/10.1016/S0020-7373(83)80031-5

Brown, W. H., Malveau, R. C., McCormick, H. W. S., & Mowbray, T. J. (1998). *AntiPatterns: refactoring software, architectures, and projects in crisis*. John Wiley & Sons, Inc.

Butler, S., Wermelinger, M., Yu, Y., & Sharp, H. (2009). Relating identifier naming flaws and code quality: An empirical study. In A. Zaidman, G. Antoniol, & S. Ducasse (Eds.), *2009 16th Working Conference on Reverse Engineering* (pp. 31–35). https://doi.org/10.1109/WCRE.2009.50

Butler, S., Wermelinger, M., Yu, Y., & Sharp, H. (2010). Exploring the influence of identifier names on code quality: An empirical study. In *2010 14th European Conference on Software Maintenance and Reengineering* (pp. 156–165). https://doi.org/10.1109/CSMR.2010.27

Caprile, C., & Tonella, P. (1999). Nomen est omen: Analyzing the language of function identifiers. In F. M. Titsworth (Ed.), *Sixth Working Conference on Reverse Engineering* (pp. 112–122). https://doi.org/10.1109/WCRE.1999.806952

Caprile, B., & Tonella, P. (2000). Restructuring program identifier names. In B. Werner (Ed.), *Proceedings 2000 International Conference on Software Maintenance* (pp. 97–107). https://doi.org/10.1109/ICSM.2000.883022

Deissenboeck, F., & Pizka, M. (2006). Concise and consistent naming. *Software Quality Journal, 14*(3), 261–282. https://doi.org/10.1007/s11219-006-9219-1

Douce, C. (2004). Metaphors we program by. In *Proceedings of the 16th Workshop of the Psychology of Programming Interest Group, Carlow, Ireland.*

Ehrlich, K., & Soloway, E. (1984). An empirical investigation of the tacit plan knowledge in programming. In J. C. Thomas & M. L. Schneider (Eds.), *Human factors in computer systems* (pp. 113–133). Ablex Publishing Corp.

Fakhoury, S., Ma, Y., Arnaoudova, V., & Adesope, O. (2018, May). The effect of poor source code lexicon and readability on developers' cognitive load. In *Proceedings of the 26th Conference on Program Comprehension* (pp. 286–296).

Fakhoury, S., Roy, D., Ma, Y., Arnaoudova, V., & Adesope, O. (2020). Measuring the impact of lexical and structural inconsistencies on developers' cognitive load during bug localization. *Empirical Software Engineering, 25,* 2140–2178. https://doi.org/10.1007/s10664-019-09751-4

Feitelson, D. G., Mizrahi, A., Noy, N., Shabat, A. B., Eliyahu, O., & Sheffer, R. (2020). How developers choose names. *IEEE Transactions on Software Engineering, 48*(1), 37–52. https://doi.org/10.1109/TSE.2020.2976920.

Fowler, M., & Beck, K. (1997, June). *Refactoring: Improving the design of existing code* [Conference presentation]. 11th European Conference. Jyväskylä, Finland.

Fry, Z. P., Shepherd, D., Hill, E., Pollock, L., & Vijay-Shanker, K. (2008). Analysing source code: looking for useful verb–direct object pairs in all the right places. *IET Software, 2*(1), 27–36.

Gil, J., & Maman, I. (2005). Micro patterns in Java code. In *Proceedings of the 20th annual ACM SIGPLAN conference on object-oriented programming, systems, languages, and applications* (pp. 97–116).

Gresta, R., & Cirilo, E. (2020, October). Contextual similarity among identifier names: An empirical study. In *Anais do VIII Workshop de Visualização, Evolução e Manutenção de Software* (pp. 49–56). SBC.

Gupta, S., Malik, S., Pollock, L., & Vijay-Shanker, K. (2013). Part-of-speech tagging of program identifiers for improved text-based software engineering tools. In H. Kagdi, D. Poshyvanyk, & M. Di Penta (Eds.), *2013 21st International Conference on Program Comprehension (ICPC)* (pp. 3–12). IEEE.

Haiduc, S., & Marcus, A. (2008). On the use of domain terms in source code. In R. Krikhaar, R. Lammel, & C. Verhoef (Eds.), *2008 16th IEEE International Conference on Program Comprehension* (pp. 113–122). IEEE.

Hochstein, L., Basili, V. R., Zelkowitz, M. V., Hollingsworth, J. K., & Carver, J. (2005). Combining self-reported and automatic data to improve programming effort measurement. *SIGSOFT Software Engineering Notes 30*(5), 356–365. https://doi.org/10.1145/1095430.1081762

Høst, E. W., & Østvold, B. M. (2007). The programmer's lexicon, volume i: The verbs. In B. Korel & M. W. Godfrey (Eds.), *Seventh IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM 2007)* (pp. 193–202).

Høst, E. W., & Østvold, B. M. (2009). Debugging method names. In S. Drossopoulou (Ed.), *European Conference on Object-Oriented Programming* (pp. 294–317). Springer Berlin Heidelberg.

Lawrie, D., Feild, H., & Binkley, D. (2006). Syntactic identifier conciseness and consistency. In M. Di Penta & L. Moonen (Eds.), *2006 Sixth IEEE International Workshop on Source Code Analysis and Manipulation* (pp. 139–148).

Lawrie, D., Feild, H., & Binkley, D. (2007a). Quantifying identifier quality: an analysis of trends. *Empirical Software Engineering, 12,* 359–388. https://doi.org/10.1007/s10664-006-9032-2

Lawrie, D., Feild, H., & Binkley, D. (2007b). An empirical study of rules for well-formed identifiers. *Journal of Software Maintenance and Evolution: Research and Practice, 19*(4), 205–229. https://doi.

org/10.1002/smr.350

Lawrie, D., Morrell, C., Feild, H., & Binkley, D. (2006). What's in a Name? A Study of Identifiers. In R. S. Bilof (Ed.), *14th IEEE international conference on program comprehension (ICPC'06)* (pp. 3–12). IEEE.

Lawrie, D., Morrell, C., Feild, H., & Binkley, D. (2007). Effective identifier names for comprehension and memory. *Innovations in Systems and Software Engineering, 3*, 303–318. https://doi.org/10.1007/s11334-007-0031-2

Liblit, B., Begel, A., & Sweetser, E. (2006, September). Cognitive Perspectives on the Role of Naming in Computer Programs. In P. Romero, J. Good, E. Acosta Chaparro, & S. Bryant (Eds.), PPIG 18 (pp. 53–67).

Newman, C. D., AlSuhaibani, R. S., Decker, M. J., Peruma, A., Kaushik, D., Mkaouer, M. W., & Hill, E. (2020). On the generation, structure, and semantics of grammar patterns in source code identifiers. *Journal of Systems and Software, 170*, 110740. https://doi.org/10.1016/j.jss.2020.110740

Ohba, M., & Gondow, K. (2005). Toward mining" concept keywords" from identifiers in large software projects. In *Proceedings of the 2005 international workshop on mining software repositories* (pp. 1–5). https://doi.org/10.1145/1083142.1083151

Reiss, S. P. (2007). Finding unusual code. In *2007 IEEE International Conference on Software Maintenance* (pp. 34–43). IEEE.

Relf, P. A. (2004). Achieving software quality through identifier names. *Qualcon 2004*, 33–34.

Relf, P. A. (2005). Tool assisted identifier naming for improved software readability: An empirical study. In L. Carvalho (Ed.), *2005 International Symposium on Empirical Software Engineering* (pp. 10–pp). IEEE.

Rilling, J., & Klemola, T. (2003). Identifying comprehension bottlenecks using program slicing and cognitive complexity metrics. In S. Kawada (Ed.), *11th IEEE International Workshop on Program Comprehension, 2003.* (pp. 115–124).

Sajaniemi, J. (2002). An empirical analysis of roles of variables in novice-level procedural programs. In B. Werner (Ed.), *Proceedings IEEE 2002 Symposia on Human Centric Computing Languages and Environments* (pp. 37–39).

Salviulo, F., & Scanniello, G. (2014). Dealing with identifiers and comments in source code comprehension and maintenance: Results from an ethnographically-informed study with students and professionals. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering* (pp. 1–10). https://doi.org/10.1145/2601248.2601251

Scanniello, G., & Risi, M. (2013). Dealing with faults in source code: Abbreviated vs. full-word identifier names. In *2013 IEEE International Conference on Software Maintenance* (pp. 190–199).

Shepherd, D., Fry, Z. P., Hill, E., Pollock, L., & Vijay-Shanker, K. (2007). Using natural language program analysis to locate and understand action-oriented concerns. In *Proceedings of the 6th international conference on Aspect-oriented software development* (pp. 212–224). https://doi.org/10.1145/1218563.1218587

Shneiderman, B. (1976). Exploratory experiments in programmer behavior. *International Journal of Computer & Information Sciences, 5*, 123–143. https://doi.org/10.1007/BF00975629

Singer, J., & Kirkham, C. (2008). Exploiting the correspondence between micro patterns and class names. In S. Kawada (Ed.), *2008 Eighth IEEE International Working Conference on Source Code Analysis and Manipulation* (pp. 67–76).

Thies, A., & Roth, C. (2010). Recommending rename refactorings. In *Proceedings of the 2nd International workshop* (pp. 1–5). https://doi.org/10.1145/1808920.1808921

Videla, A. (2017). Metaphors we compute by. *Communications of the ACM, 60*(10), 42–45. https://doi.org/10.1145/3106625

Von Mayrhauser, A., & Vans, A. M. (1998, June). Program understanding behavior during adaptation of large scale software. In K. Kelly (Ed.), *Proceedings. 6th International Workshop on Program Comprehension. IWPC'98* (Cat. No. 98TB100242) (pp. 164–172).

RECEIVED 24.10.2022 | ACCEPTED 15.5.2023